Wind to Power Modeling - Assignment 1

Peter Henrik George Mitchell
IT-University of Copenhagen
pemi@itu.dk

Course Title: Large Scale Data Analysis

Course Code: BSLASDA1KU
Repository: https://github.com/Peter-mitch1/pemi_LSDA/tree/main

March 15, 2026

1 Introduction

1.1 Context

This analysis focuses on the transformation and modeling of wind data to predict power generation. The study culminates in the implementation of regression models optimized to predict power output using wind speed and directional vectors as primary features. In the following sections and subsections each step to build the predictive models is given. **The models featured in this Report are Decision Tree Regressor's and Support Vector Regression's**.

2 Data Alignment

time	Direction	Lead_hour	Source_tin	Speed
2021-12-11 15:00:00+00:00	SSE	1	1.64E+09	11.176
2021-12-11 18:00:00+00:00	SSW	1	1.64E+09	8.04672
2021-12-11 21:00:00+00:00	WSW	1	1.64E+09	11.176

Figure 1: weather.csv

time	ANM	Non-ANM	Total
2021-12-11 13:39:00+00:00	10.19556	17.102	27.29756
2021-12-11 13:40:00+00:00	10.04356	17.582	27.62556
2021-12-11 13:41:00+00:00	10.96156	17.139	28.10056
2021-12-11 13:42:00+00:00	11.24056	16.194	27.43456

Figure 2: power.csv

The crux of this assignment's problem was formatted around a bias-variance trade-off of how to merge data of different granularities. The two csv's readings were taken at different time intervals. The `weather.csv` readings were taken every three hours. The `power.csv` readings were taken every minute. This presented a situation of potential underfitting or overfitting.

```
#Downsampling#
# 1. Setup the data

# Load data
df_power_agg = power_df

# Ensure the time index is set
# Starting at 2021-12-11 13:39:00+00:00
df_power_agg.index = pd.date_range(
    start="2021-12-11 13:39:00+00:00",
    periods=len(df_power_agg),
    freq='1min'
)

# Resample every 3 hours ('3H') with an origin at 15:00
resampled_df = df_power_agg.resample(
    '3h',
    origin='2021-12-11 15:00:00+00:00',
    closed='right',
    label='right'
).mean()

df_downsampled = resampled_df.join(wind_df, how='inner')
df_downsampled
```

Figure 3: Downsample code implemented in Jupyter Notebook

```
# Sort by 'time' (required for merge_asof)
power_df = power_df.sort_values('time')
wind_df = wind_df.sort_values('time')

# Perform "as-of" join
# 'backward' matches each power entry with the weather recorded AT or BEFORE that time.
merged_df = pd.merge_asof(power_df, wind_df, on='time', direction='backward')

upsampled_df = merged_df.dropna()
```

Figure 4: Upsample code implemented in Jupyter Notebook

Two approaches were used to explore this trade-off, decimation(downsampling) and interpolation(upsampling). I prefer the terms `downsample` and `upsample`, I'll be referring to them us such from now on. `Downsampling` was used first and then `upsampling` was utilised retroactively given the poor results that the `downsampled` data ended up yielding.

Worth mentioning is the incurred data loss and synthetic data generated. The data loss for merging through `downsampling`, `power.csv` was reduced from 109026 to 604, and `weather.csv` reduced from 716 to 604. The intended advantage of `downsampling` was to keep the data 'real'. The `Total` power was encoded temporally presenting a mean average of the value over the 3 hour period. However in hindsight this might have taken away from how 'real' the data was.

For the `upsampling` we have an inverted problem of statically creating 108,310 synthetic entries for the weather data. As such it puts into question how real the data itself the model is training on actually is and whether that is okay?

3 Data pre-processing and pipelines

3.1 Data-splitting strategy

```
test_size = int(len(df_engineered)*0.15)
train_size = len(df_engineered) - test_size

trainingDF = df_engineered.iloc[:train_size]
finalTestDF = df_engineered.iloc[train_size:]

print(f"Training set: {len(trainingDF)} rows")
print(f"Test set: {len(finalTestDF)} rows")
```

Figure 5: Code to withold the final test data set.

The `.dropna()` function was used consistently throughout the notebook to ensure no missing value columns permeated the dataset. As seen in `Figure 5` I removed a conservative portion of 15% for the final test that the data is not trained on. This 15% choice stems from the lack of data in the downsampled dataframe which had only 604 entries. As such trying to keep as much for training was preferred. This same code was also used for the upsampled dataframe.

The default values in the `train_test_split` function were used initially for model exploration the split defaults to 75% for training and 25% for testing.

To quickly test the values and potential models the data was first used without Time-Series implemented. I should have ensured cross validation throughout but I simply forgot. However I do implement cross validation on the final model (RT_Final).

3.2 Converting Wind Directions

```
class WindDirectionEncoder(BaseEstimator, TransformerMixin):
    def __init__(self, column_name='Direction'):
        self.column_name = column_name
        self.mapping = {
            'N': 0, 'NNE': 22.5, 'NE': 45, 'ENE': 67.5,
            'E': 90, 'ESE': 112.5, 'SE': 135, 'SSE': 157.5,
            'S': 180, 'SSW': 202.5, 'SW': 225, 'WSW': 247.5,
            'W': 270, 'WNW': 292.5, 'NW': 315, 'NNW': 337.5
        }

    def fit(self, X, y=None):
        # Nothing to learn from the data here, so we just return self
        return self

    def transform(self, X):
        # Works on a copy to prevent modifying the original dataframe
        X = X.copy()
        # 1. Checks if the data has already been transformed
        if 'dir_sin' in X.columns and 'dir_cos' in X.columns:
            # If the original column is still there, drop it; otherwise just return X
            if self.column_name in X.columns:
                return X.drop(columns=[self.column_name])
            return X

        # 2. Safety check: does the source column actually exist?
        if self.column_name not in X.columns:
            raise KeyError(f"The column '{self.column_name}' is missing from the input features.")

        # 3. Perform mapping and cyclical encoding
        deg = X[self.column_name].map(self.mapping).fillna(0)
        X['dir_sin'] = np.sin(2 * np.pi * deg / 360)
        X['dir_cos'] = np.cos(2 * np.pi * deg / 360)

        # 4. Drop the original categorical column
        return X.drop(columns=[self.column_name])
```

Figure 6: class built for WindDirectionEncoder to add to pipeline.

`class WindDirectionEncoder` was built to map the cardinal directions into usable features. Cyclical encoding is used to map categoricals to their corresponding degree, then these are converted into two new features of `'dir_sin'`(sine) and `'dir_cos'`(cosine). What this ensures is that the features are standardised and that the spatial relationship, for instance, `N: 0` and `NNW: 337.5` are close, is correctly retained.

3.3 Speed Features Engineered

```python
# Create trailing 24-hour moving average
df_downsampled['Speed_MA_24h'] = df_downsampled['Speed'].rolling(window=8).mean()

# Create trailing 24-hour standard deviation (captures volatility)
df_downsampled['Speed_STD_24h'] = df_downsampled['Speed'].rolling(window=8).std()

# Create trailing 3-day moving average (longer trend)
df_downsampled['Speed_MA_3d'] = df_downsampled['Speed'].rolling(window=24).mean()

# Rolling windows create NaN values
df_engineered = df_downsampled.dropna()
```

Figure 7: Speed features engineered to add additional smoothing.

Depending on implementation rolling averages can provide higher variance or bias. In the instance of the features introduced here it may be suggested it adds higher variance. Also of note is that the downsampled models performed poorly and the engineered speed features likely only added more noise. Given this no rolling average was used on the upsampled data. Though it may have made sense with a larger exposure window.

3.4 Pipeline Builds

```python
regressionTree_pipeline = Pipeline([
    ('Wind_Encoding', WindDirectionEncoder(column_name='Direction')),
    ('imputer', SimpleImputer(strategy='median')), # Preprocessing
    ('regressor', DecisionTreeRegressor(max_depth=5, random_state=42)) # Model
])
✓ 0.0s

svr_pipeline = Pipeline([
    ('Wind_Encoding', WindDirectionEncoder(column_name='Direction')),
    ('imputer', SimpleImputer(strategy='median')), # Fill missing values
    ('scaler', StandardScaler()),                  # CRITICAL: Scale to mean=0, var=1
    ('regressor', SVR(kernel='rbf', C=1.0, epsilon=0.1))
])
✓ 0.0s
```

Figure 8: Pipeline for two Regressors - Regression Trees and Support Vector Regression.

Pipeline was fairly straight forward, the code in `Figure 6` was added to the pipeline labeled `WindDirectionEncoder()`, alongside `StandardScaler()`, `SimpleImputer()`(another NaN mitigation tool) and the specific choice of regressor model looking to be used. The scaler was omitted for Regression trees with the thought that unscaled data would help with making splits. This pipeline build would remain consistent throughout even with the introduction of Time Series.

3.5 Optuna Optimisation

I used Optuna an open-source API that helps tune hyperparameters. It does so through Bayesian Optimisation. I defined 200 runs for the SVR's (Support Vector Regression) models and 500 runs for Decision Tree Regressor. The goal of which was to minimise Mean Absolute Error through Bayesian Optimisation. This method worked quite well for the Decision Trees, however for the SVR's it did not perform quite as well. The final model was RT_Final (Decision Tree Regressor) with the following set hyperparameters chosen from 500 runs using Optuna. `DecisionTreeRegressor(max_depth=4, min_samples_split=15, min_samples_leaf=18, max_features= None, random_state=42,))`

4 Model training and evaluation

4.1 Results

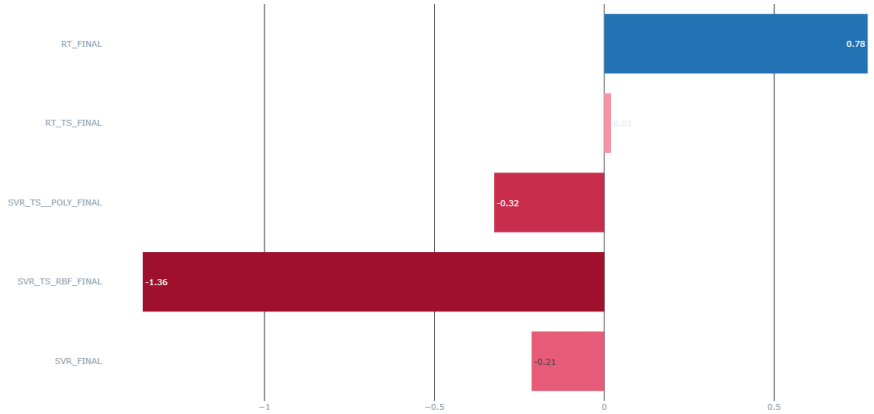

Figure 9: MLFlow Bar chart of Adjusted R-Squared Values of All Final Models

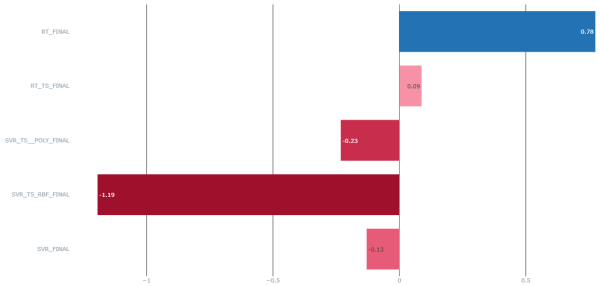

Figure 10: MLFlow Bar chart of R-Squared Values

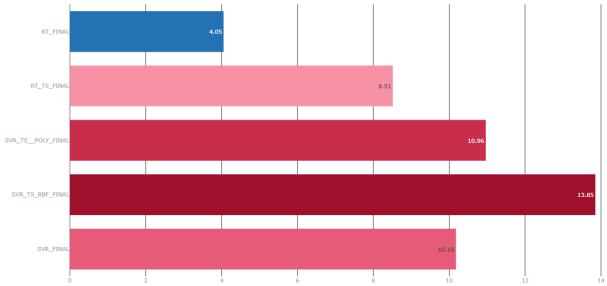

Figure 11: Mean Absolute Error of Final Models

Here we can see the final results of the modeling. All model's marked in red are Downsampled. The extremely poor performance of which led to deciding to Upsample and create the model marked in blue RT_Final an Upsampled Decision Tree Regressor. As is quite clear the upsampled data based on R-Squared metrics and Mean Absolute Error performs significantly better. Nearing towards 80% feature explainability and a Mean Absolute Error of 4.05.

What makes this of note is that RT_Final was only trained on the upsampled data with no feature engineering of Speed and only a static cyclical encoding of Direction. In `Figure 12` we can see this fit over the time series data.

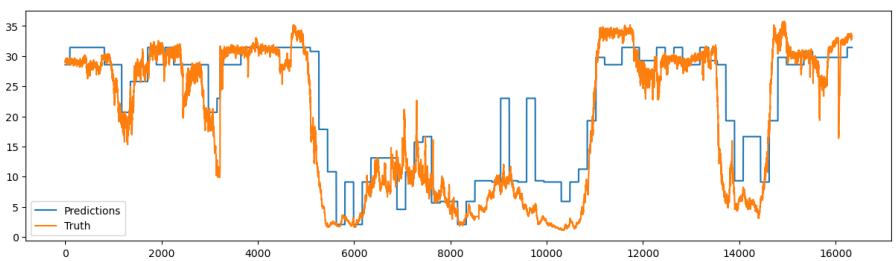

Figure 12: RT_Final Imaged Performance.

As we can see from `Figure 12` `RT_Final` learnt the trends and patterns in the data. Which can not be said for the other models. Also worth mentioning that while the size of the data increased the distribution itself remained the same.

5 Discussion

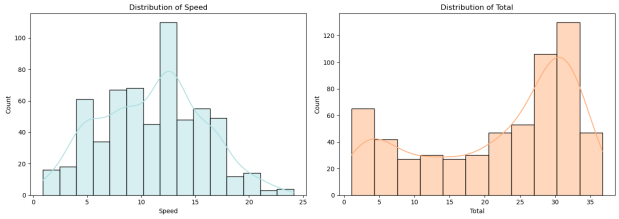

Figure 13: Downsampled Distribution

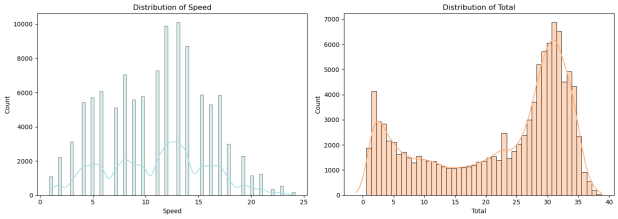

Figure 14: Upsampled Distribution

My naive interpretation of `Figure 13` and `Figure 14` would be that overall upsampling retains the general distribution of the data. Providing far more training points and leading to more model efficacy.

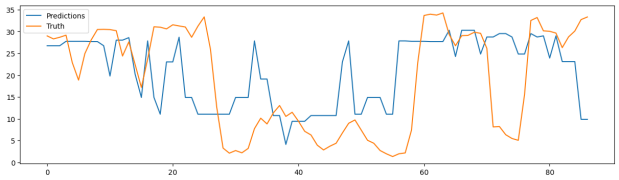

Figure 15: Decision Tree Regressor on Downsampled Time-Series Data

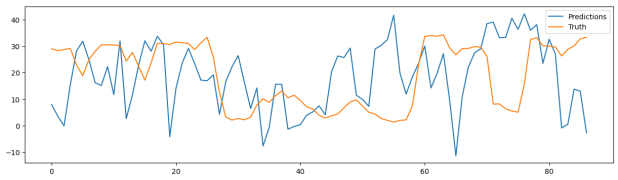

Figure 16: Support Vector Regression on Downsampled Time-Series Data

I think the largest limitation by far is the exposure window of the initial recordings. 3 months of Data is clearly not enough data to help with modelling without compromising in some respect. Above in `Figure 15` and Figure 16 show the downsampled trials. These models show an in-ability to learn any meaningful trends in 3 hour increments of 90 days. Therefore when it comes to granularity from small empirical evidence of this study it would be suggested to prefer upsampling over downsampling. This is also highlighted by the results themselves.

Also interesting is the explainability of the target variable from only three features we achieved a 77% Adjusted R-Squared with no dip from regular R-Squared indicating that all features inputted remain important in the modeling of the target variable.

What this tells us is that an engineered feature does not equal a good feature. Likely good features are present from the start and have a strong relationship to begin with. It is much harder to generate or engineer such a thing. This does not mean that engineered features cannot work, just that their effect in this study was marginal.

6 Conclusion

What would have been better to have now and into the future would be better time-series implementation where granularity of the data from inception is matched. The project was packaged and served and is thus available to trial and test these results. Only the final model was implemented in code to make it concise and clear.